

Need for OCSF, Semantic Layer, and Central Catalogue

Cyber data platform view - purpose, with/without comparison, and where each is useful

Executive summary

OCSF, semantic layer, and central catalogue are not the same thing. They work together to make cyber data usable, governed, searchable, and trustworthy across domains.

Raw source data -> OCSF normalized data -> Curated cyber entities -> Semantic layer -> Central catalogue -> Dashboards / Agents / NLQ / Governance

Capability	Main purpose	Simple answer
OCSF	Standardize security records	Common language for events, findings, resources, users, activities, severity, and status.
Semantic layer	Standardize business/cyber meaning	Common definitions for Asset, Application, Risk, Owner, Exposure, SLA breach, and reusable relationships.
Central catalogue	Standardize discovery, metadata, ownership, lineage, and trust	A searchable inventory of data products, schemas, semantic entities, metrics, owners, policies, and quality signals.

1. Why OCSF is needed

Different cyber tools describe the same thing differently. OCSF reduces tool-specific mapping by normalizing records into a common security schema.

Without OCSF	With OCSF
Tenable, Qualys, CrowdStrike, Wiz, AWS, Azure, AD, and SIEM logs all use different field names and categories.	Security records are mapped to common event/finding classes and common field structures.
Queries need many source-specific mappings, unions, and exception rules.	Queries and detections can use normalized concepts such as resource, actor, endpoint, severity, activity, status.
New source onboarding breaks dashboards and downstream logic.	New source onboarding mainly requires source-to-OCSF mapping; downstream consumers change less.
Detection logic becomes vendor-specific.	Detection and analytics logic becomes more portable across vendors.
Analysts and agents need to understand every tool schema.	Analysts and agents can reason over a common cyber record model.

Where OCSF is useful

- Security event normalization: authentication, network activity, endpoint activity, cloud activity, IAM activity.
- Security findings normalization: vulnerability findings, compliance findings, detection findings, configuration findings.
- Cross-tool analytics where multiple vendors produce similar security signals.
- Reusable detections, dashboards, and data pipelines across domains.
- A stable silver layer between raw logs and curated business/cyber entities.

2. Why semantic layer is needed

OCSF tells you what the record is. The semantic layer tells you what the record means in your enterprise cyber context.

Without semantic layer	With semantic layer
Users must know physical tables, columns, and join paths.	Users ask using concepts such as Application, Asset, Owner, Risk, Vulnerability, Control, Exposure.
Metrics are duplicated differently across dashboards and teams.	Metrics are centrally defined, versioned, and reused.
Agents may guess wrong joins or wrong definitions.	Agents use approved semantic definitions and relationship paths.
Risk, ownership, criticality, SLA, and exposure logic is hidden in SQL.	Risk, ownership, criticality, SLA, and exposure logic is explicit and governed.
Every domain builds its own meaning layer.	Common cyber/business concepts are shared while allowing domain-specific extensions.

What the semantic layer defines

Semantic object	Examples
Entities	Asset, Application, Vulnerability, Identity, Certificate, Control, Finding, Cloud Account, Business Unit.
Relationships	Application has assets; Asset has vulnerabilities; Identity has access; Asset belongs to cloud account; Control mitigates risk.
Metrics	Critical vuln count, internet-facing asset count, SLA breach count, expired cert count, application risk score.
Rules	Authoritative owner source, active asset definition, critical asset definition, exposure definition, risk scoring logic.
Policies	Tenant scope, column access, data sensitivity, allowed join paths, row-level filters.

Where semantic layer is useful

- NL2SQL and agentic workflows where the user asks business/cyber questions instead of table-specific questions.
- Dashboards that need consistent metrics across domains and executive reporting.
- Cross-domain use cases such as vulnerability chaining, PKI expiry, threat modeling, identity exposure, and application risk.
- Governed joins between asset inventory, CMDB, vulnerability data, IAM, cloud exposure, and controls.
- Reusable policy logic so each team does not implement its own interpretation of risk or ownership.

3. Why a central catalogue is needed

A central catalogue is the discovery and governance layer for data and semantics. It does not need to hold all domain data. It should hold metadata, data product definitions, schemas, lineage, ownership, quality signals, policy tags, and semantic mappings.

Without central catalogue	With central catalogue
Teams do not know which data exists, which table is trusted, or who owns it.	Teams can search approved data products, schemas, owners, SLAs, freshness, quality, and access rules.
Multiple versions of asset, app, vulnerability, and owner data appear across projects.	Catalogue marks source-of-truth datasets and approved semantic definitions.
Agents and analysts discover tables by guessing or tribal knowledge.	Agents and analysts use catalogue metadata to choose the right datasets and concepts.
Governance is scattered across project docs, SQL, tickets, and emails.	Governance metadata is centralized: owner, classification, policy, lineage, certification, usage.
Data onboarding lacks visibility.	New source onboarding creates catalogue entries, mappings,

	lineage, and quality checks.
--	------------------------------

What the central catalogue should contain

Catalogue area	Purpose	Examples
Data products	Discover approved datasets and APIs.	vulnerability_findings_gold, asset_inventory_gold, iam_entitlements, application_risk_daily.
Schema metadata	Understand fields and types.	resource_uid, tenant_id, severity_id, app_id, owner_group, data_classification.
Semantic definitions	Expose business/cyber concepts.	Critical Asset, Internet-Facing, Crown Jewel, Active Vulnerability, SLA Breach.
Ownership and stewardship	Know who owns and supports data.	Data owner, technical owner, domain owner, escalation contact, SLA.
Lineage	Trace where data came from and where it is used.	Tenable raw -> OCSF vuln finding -> gold vulnerability -> risk dashboard.
Quality and freshness	Build trust in the data.	Last refresh, completeness, duplicates, failed mappings, stale asset count.
Policies and access	Apply governance and safe query boundaries.	Tenant rules, PII tags, restricted columns, approved use cases.

Where central catalogue is useful

- Data discovery: users and agents can find the right dataset without asking platform engineers.
- Governance: ownership, classification, lineage, certification, and access policies are visible centrally.
- Agent grounding: NL2SQL agents can use catalogue metadata to select trusted tables and semantic definitions.
- Domain project separation: each domain can own data in its GCP project while central catalogue stores metadata and pointers.
- Auditability: teams can answer who owns the data, how it was produced, where it is used, and whether it is trusted.

4. With and without OCSF + semantic layer together

Scenario	What happens	Main issue / benefit
No OCSF, no semantic layer	Raw vendor schemas plus raw business tables. Every query is custom.	Hardest to scale. Inconsistent analytics, fragile agents, repeated joins, poor governance.
OCSF only	Security records are normalized, but business meaning is still missing.	Good for event/finding consistency, but not enough for risk, ownership, app context, and NLQ.
Semantic layer only	Business definitions exist, but source mappings are messy because raw tools are inconsistent.	Can work, but requires many custom adapters and high maintenance.
OCSF + semantic layer	Security records are normalized and mapped into governed enterprise cyber concepts.	Best path for reliable dashboards, NL2SQL, agents, cross-domain analytics, and governed risk reporting.
OCSF + semantic layer + central catalogue	Normalized data, governed meaning, and searchable metadata are connected.	Best enterprise operating model: discoverable, trusted, reusable, auditable, and easier to onboard new sources.

5. Recommended enterprise architecture

Domain GCP projects hold domain-owned data products

- Cyber asset project
- Vulnerability project
- IAM / AD project
- Cloud security project
- PKI project
- Threat / detection project

Central catalogue project holds metadata and semantics

- Data product registry
- OCSF mapping registry
- Semantic entity and metric registry
- Ownership, lineage, certification, and quality metadata
- Policy tags and access rules
- Pointers to BigQuery tables, graph nodes/edges, APIs, and dashboards

Query / agent layer uses catalogue + semantic layer

- Discover trusted data products
- Resolve business terms to semantic definitions
- Apply tenant and policy rules
- Generate SQL, graph query, or API calls against domain-owned sources

6. Example: user asks a cyber business question

Question: Show applications with critical vulnerabilities on internet-facing assets and missing ownership.

Layer	Role in answering the question
Central catalogue	Find trusted data products for applications, assets, vulnerability findings, exposure, and ownership. Confirm owners, freshness, and certified datasets.
Semantic layer	Resolve terms: application, critical vulnerability, internet-facing asset, missing ownership. Provide approved relationships and metrics.
OCSF layer	Provide normalized vulnerability finding records with consistent severity, status, resource, and finding structure.
Domain data products	Execute against domain-owned BigQuery tables, graph edges, or APIs.
Governance layer	Apply tenant scope, access control, sensitive columns, and allowed data usage.

7. Ownership model

Area	Best owner	Responsibility
Raw and domain data products	Domain data teams	Ingest, validate, own SLAs, publish certified data products.
OCSF mappings	Security data engineering + domain SMEs	Map source data to OCSF classes and fields; maintain mapping quality.
Semantic layer	Central ontology/semantic team + domain SMEs	Define enterprise concepts, metrics, joins, source-of-truth rules, and domain extensions.
Central catalogue	Data governance / platform team	Manage metadata registry, lineage, ownership, quality, certification, and access metadata.
Graph / ontology relationships	Ontology/graph platform + domain teams	Maintain cross-domain relationships and approved relationship models.
Consumers	Analytics, dashboard, and agent teams	Use approved catalogue entries and semantic definitions instead of hardcoding raw table logic.

8. Practical takeaway

Use OCSF for normalization, semantic layer for meaning, and central catalogue for discovery and governance. OCSF makes data consistent. The semantic layer makes it understandable. The central catalogue makes it findable, trusted, owned, and reusable.

Need	Best fit
Normalize vendor-specific cyber data	OCSF
Define enterprise cyber/business meaning	Semantic layer
Discover trusted datasets and APIs	Central catalogue
Track owner, freshness, quality, and lineage	Central catalogue
Answer business questions like risky apps, exposed assets, SLA breach, crown-jewel risk	Semantic layer + OCSF + catalogue
Support multi-domain GCP project model without centralizing all raw data	Central catalogue with pointers to domain-owned data products